

Enterprise Agentic AI Platform — Ownership & Shared Responsibility

Infrastructure Services (IS) vs the AI Platform — who owns what on AWS Bedrock AgentCore

June 23, 2026

AWS Bedrock AgentCore is a managed **substrate** — a runtime plus a set of primitives (memory, gateway, identity, observability) — the same way RDS, EKS, or a Kafka cluster is a substrate. Provisioning that substrate is necessary but **not the same as having an enterprise platform**. This note clarifies what **IS owns** versus what the **AI Platform team owns**, where their responsibilities meet (the *seams*), the two distinct kinds of "guardrails," and a RACI to settle ownership.

Roles in this document

Role	Who they are
IS — Infrastructure Services	Provisions and operates the underlying AWS environment and managed services; owns the secured, compliant substrate.
AI Platform team	Builds the governed platform on top of the substrate (registry, orchestration, policies, tools, evaluations, console) that application teams build on.
App / product teams	Build and own the specific use-case agents (e.g. claims, underwriting) <i>using</i> the platform — the platform's tenants/consumers, not its builders.
Risk / Compliance	Sets and signs off on the regulatory rules the system must obey (PII handling, approval requirements, audit, denied actions). Accountable, not a builder; includes Legal / Security / Data Governance.

1. The layered ownership model

Layer	Owns	Delivers
Infrastructure Services (IS)	Account/landing zone, VPC & networking, IAM baseline + infrastructure guardrails (§4); provisions and operates the managed services (AgentCore Runtime, Bedrock access, Gateway/Memory infra, Fargate); patching, quotas, SLAs, cost plumbing	"A secured, compliant, provisioned AgentCore + Bedrock environment"
AI Platform team	Agent blueprints + registry/lifecycle (create/reuse/clone), orchestration patterns , the policy repository + AI guardrail content (§4), memory strategies , the tool/Gateway catalog + agents-as-tools, RAG curation , eval/safety harness, observability semantics + audit , developer experience (templates, CI/CD), the console/UI	"A paved road that app teams safely build agents on"
Application / product teams	The actual use-case agents (e.g. claims adjudication, underwriting) built using the platform	Business outcomes

AgentCore gives you *primitives*; none of them give you your blueprint model, governance rules, curated tool catalog, evaluations, or developer experience. Those are the platform.

2. Is it difficult to separate IS out? — No, if the line is drawn deliberately

The interfaces are clean. AgentCore exposes a **control plane** (provision/configure — IS, via Infrastructure-as-Code) versus a **data plane** (invoke/use — Platform & apps). IS owns provisioning + infrastructure guardrails; the Platform owns everything configured and built on top. It only becomes hard when **nobody draws the line** — which is the ambiguity this document removes.

3. Where ownership / handover / handshaking gets confused (the seams)

A managed service like AgentCore **collapses several traditional layers** (compute + identity + memory + gateway) into one product — which is why it can look like "it's all in there." Responsibility still splits, at predictable seams that must be assigned explicitly:

Seam	The ambiguity	Clean split
Identity	IS owns account IAM; who mints per-agent workload identities + downstream permissions?	IS: baseline + guardrails. Platform: agent identities & credential vending. System owner: approves access
Gateway / tool exposure	Who decides which enterprise API becomes an MCP tool, and approves an agent calling it?	IS: network path. Platform: tool definition + authorization policy. Data owner: approval
Memory & data	Who owns retention, PII classification, right-to-be-forgotten?	IS: storage/encryption. Platform: memory strategy + retention. Data governance: PII rules
Observability	Who owns the compliance audit trail versus the raw logs?	IS: CloudWatch/log infrastructure. Platform: trace semantics, dashboards, audit reports
FinOps	Consumption-priced; who owns per-agent/per-tenant cost attribution + budgets?	Platform (with IS tagging support); Finance accountable
CI/CD	Who owns the agent build → deploy pipeline (CodeBuild → ECR → AgentCore)?	Platform, on IS-provided primitives

4. Two kinds of "guardrails" (a common point of confusion)

The word "guardrails" is overloaded — two different layers, owned by two different teams. This is usually the single biggest source of the ownership argument.

Dimension	Infrastructure / landing-zone guardrails	AI / behavioral guardrails
What it is	Controls on what can be provisioned and how the environment behaves	Controls on how the agent/model behaves
Examples	SCPs, AWS Config rules, IAM permission boundaries, network/egress controls, encryption-at-rest, hardened base/container images (golden images, CIS benchmarks)	Bedrock Guardrails content (denied topics, content filters, PII redaction in model I/O , prompt-injection defense), tool-authorization, human-approval gates, the policy repository
Owner	IS	Platform team (+ Risk / Compliance)

*The subtlety — **Bedrock Guardrails splits ownership**: IS may enable the capability in the account, but the Platform team and Risk/Compliance author the guardrail content (which topics are denied, which PII is masked, which actions need approval). **Enablement is not authorship**. So when IS "owns guardrails," that means the **infrastructure baseline and base images** — **not** the platform-level AI guardrails, which the Platform team owns.*

5. RACI — ownership by responsibility, not by service

The resolving principle: **assign ownership by responsibility, not by service** — the way cloud providers publish a shared-responsibility model. Each capability gets a named owner, which dissolves the "it's all AgentCore, so it's all

done / all ours" claim.

R = Responsible · A = Accountable · C = Consulted · I = Informed

Capability	IS	Platform	App	Risk/Comp
Account / network / landing zone	A,R	I	I	C
AgentCore & Bedrock provisioning	A,R	C	I	I
Infra guardrails (SCP/Config/base images/encryption)	A,R	C	I	C
Agent identity & credential vending	C	A,R	I	C
Tool exposure (Gateway targets)	C	A,R	C	C
Agent blueprints / registry / lifecycle	I	A,R	C	I
Orchestration patterns	I	A,R	C	I
Memory strategy & retention	C	A,R	I	C
AI / behavioral guardrail content	C	R	I	A
RAG / knowledge-base curation	I	A,R	C	C
Observability semantics & audit	C	A,R	I	C
FinOps / cost attribution	C	R	I	A=Finance
CI/CD for agents	C	A,R	C	I
Evaluation / safety / quality harness	I	A,R	C	C
Use-case agents	I	C	A,R	C

6. Bottom line

- Separation is **achievable and not technically hard** — clean control-plane vs data-plane interfaces.
- The confusion is **organizational, not architectural**, and is **predictable** at the named seams.
- AgentCore being present means the **substrate** is there; the **platform** is the governed layer the AI Platform team owns on top.
- Resolve it once with a **shared-responsibility model + RACI**, and the ownership debate goes away.